

~ Ratio, Log Transforms and Domain-Specific Features

```
import numpy as np
df['goals_per_90'] = df['goals'] / df['minutes'] * 90
df['xG_per_shot'] = df['xG'] / df['shots']
df['log_salary'] = np.log1p(df['salary']) # log transforms
```

2 usul:

High-Order: ko'proq feature yaratamiz eski ham goladi
 Ratio, Log Transform: eski ko'p holatlarda qolmaydi, fagatgina yangisi bilan
 holdiramiz
 Inxida ham matematik amallar ishlatamiz ekan.

Target Encoding with Leakage Prevention

- Categorical qiymatlar uchun $\Rightarrow$ mean of target
 (Analiz)

```
from category_encoders import TargetEncoder
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ("target_enc", TargetEncoder(cols=['position'])),
    ("model", RandomForestRegressor())
])

scores = cross_val_score(pipeline, X, y, cv=5)
print(scores)
```

Masalan: • Log Transformation / • Domain (xattalix o'g'ir mahiyat)

Frequency / Count Encoding (Categorical Feature)

```
# Analiz Frequ encoding
freq = df['club'].value_counts(normalize=True)
df['club_freq'] = df['club'].map(freq)
```

Target and Frequency Encoding

Date.

No.

• Overfitting vs Leakage

- **Overfitting** $\Rightarrow$ data ^{trainida yaxshi testda yomon chiqadi} noto'g'ri ma'lumotlarni ko'p o'rganish
(Yodlab olingan o'quvchi $\Rightarrow$ yangi ma'lumot yoni testni ishlamay olmaydi)
- **Leakage** $\Rightarrow$ to'g'ri data lekin trainingda bolmasligi kerak bo'lgan ma'lumotlarni ko'p o'rganish
(Intixon savollarini intixondan oldin krib yodlab olingan o'quvchi)

! Target qiymatdan feature yaratishda foydalanmang!!

- Leakage / - Classlardagi kashollik

• Time-Based Rolling Features and Lag Features

- Time-series yoni performance dan foydalanishda koldan featurelar yaratish
- Lag 1 = previous game's xG
- Lag 3 = average of last 3 games

Example: last 5 matches / performance trends / consistency ^{uzaytirish}

* Time based Rolling and Lagging

$df = df.sort_values(['player_id', 'match_date'])$

$df['xG_lag1'] = df.groupby('player_id')['xG'].shift(1)$

$df['xG_rolling3'] = df.groupby('player_id')['xG'].rolling(3).mean().reset_index(reset_index(0, drop=True))$